Case Study 5: Review Sentiment

Aim: classify Amazon product review text as negative, neutral or positive; fine-tune a HuggingFace pretrained BERT, compare an LSTM baseline, and discuss transfer learning efficiency and deployment constraints.

Source: Keung, Lu, Szarvas and Smith (2020), The Multilingual Amazon Reviews Corpus, English subset. https://arxiv.org/abs/2010.02573. Downloaded via https://huggingface.co/datasets/SetFit/amazon_reviews_multi_en (mirror metadata: Apache-2.0). No synthetic reviews are included.

Data: 6,300 reviews; test=900; train=4,500; validation=900. Each split is balanced across three sentiments. Within negative/positive, both associated star ratings are sampled equally. Sampling seed: 42.

Original train/validation/test membership is preserved. Duplicate normalized review text is removed across splits. Rating-derived labels: 1/2 stars negative; 3 neutral; 4/5 positive. Stars never enter either model. A three-star review may express mixed feelings rather than true neutrality.

BERT: google/bert_uncased_L-4_H-256_A-4. Genuine Google pretrained BERT-Mini, four layers and 256 hidden units; every parameter is fine-tuned for 4 epochs, AdamW learning rate 5e-5. This experiment does not benchmark BERT-Base.

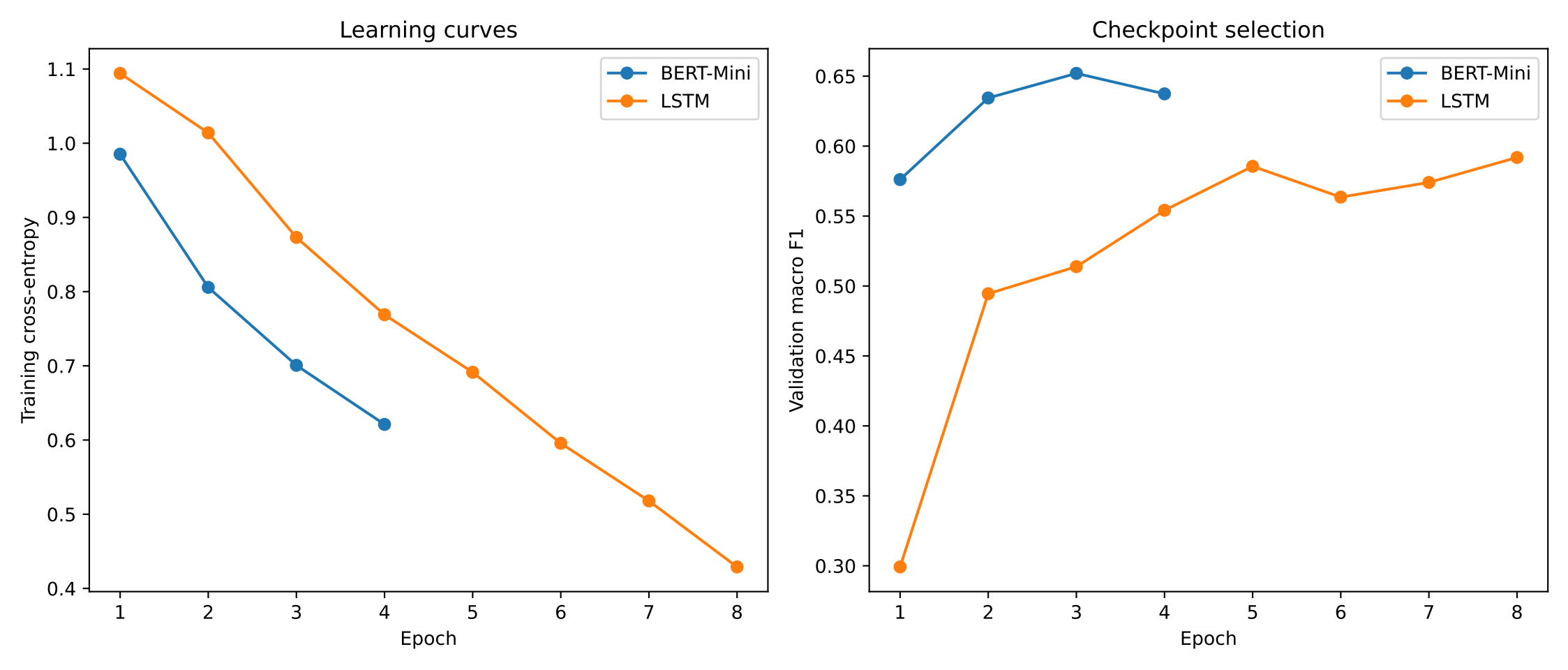
LSTM: random 64-dimensional embeddings -> one 64-unit LSTM -> linear 3-class head; 8 epochs, AdamW learning rate 1e-3. Same fixed WordPiece tokenizer as BERT, but no pretrained embedding weights.

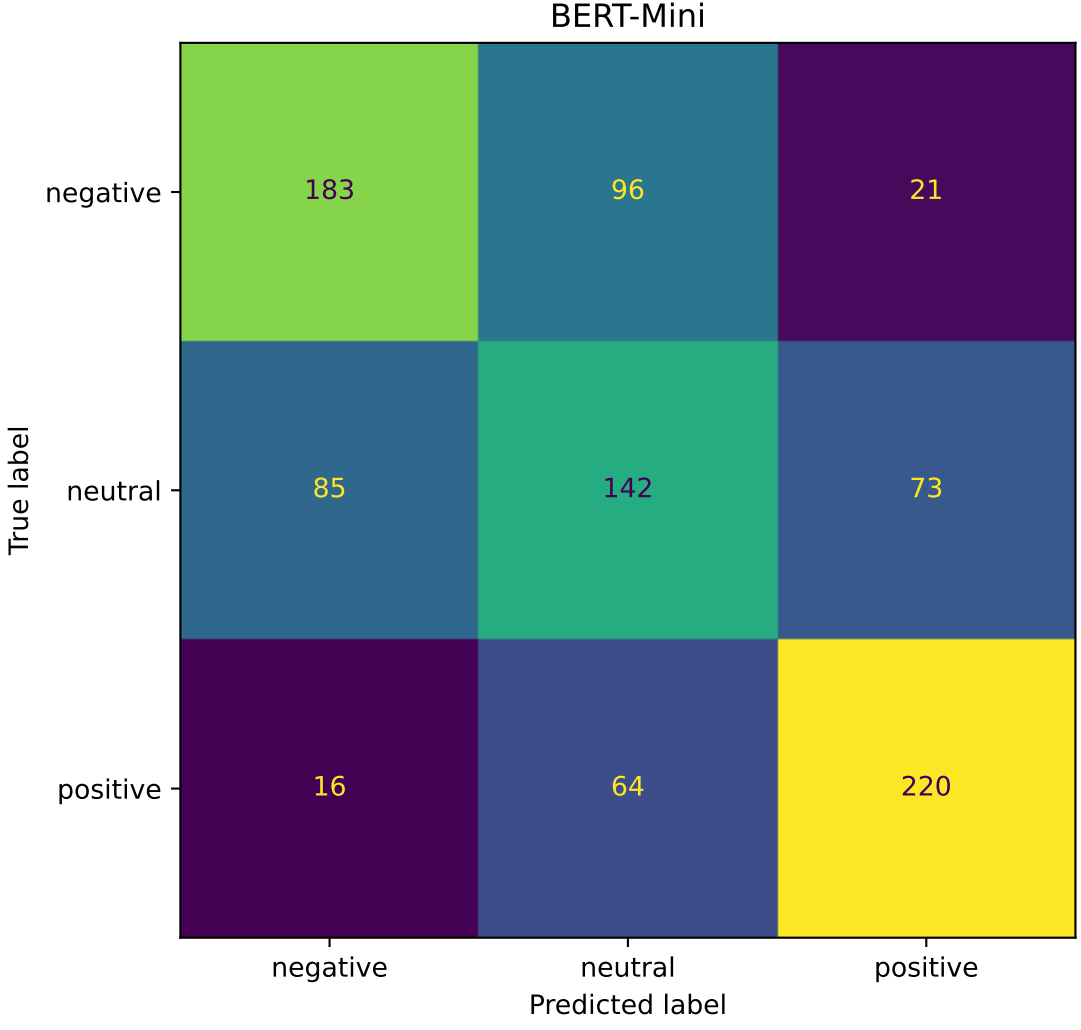
Both: maximum 96 tokens, batch size 32, cross-entropy loss, weight decay 0.01, gradient clipping 1.0. Best epoch is selected using validation macro F1; the held-out test set is evaluated afterward.

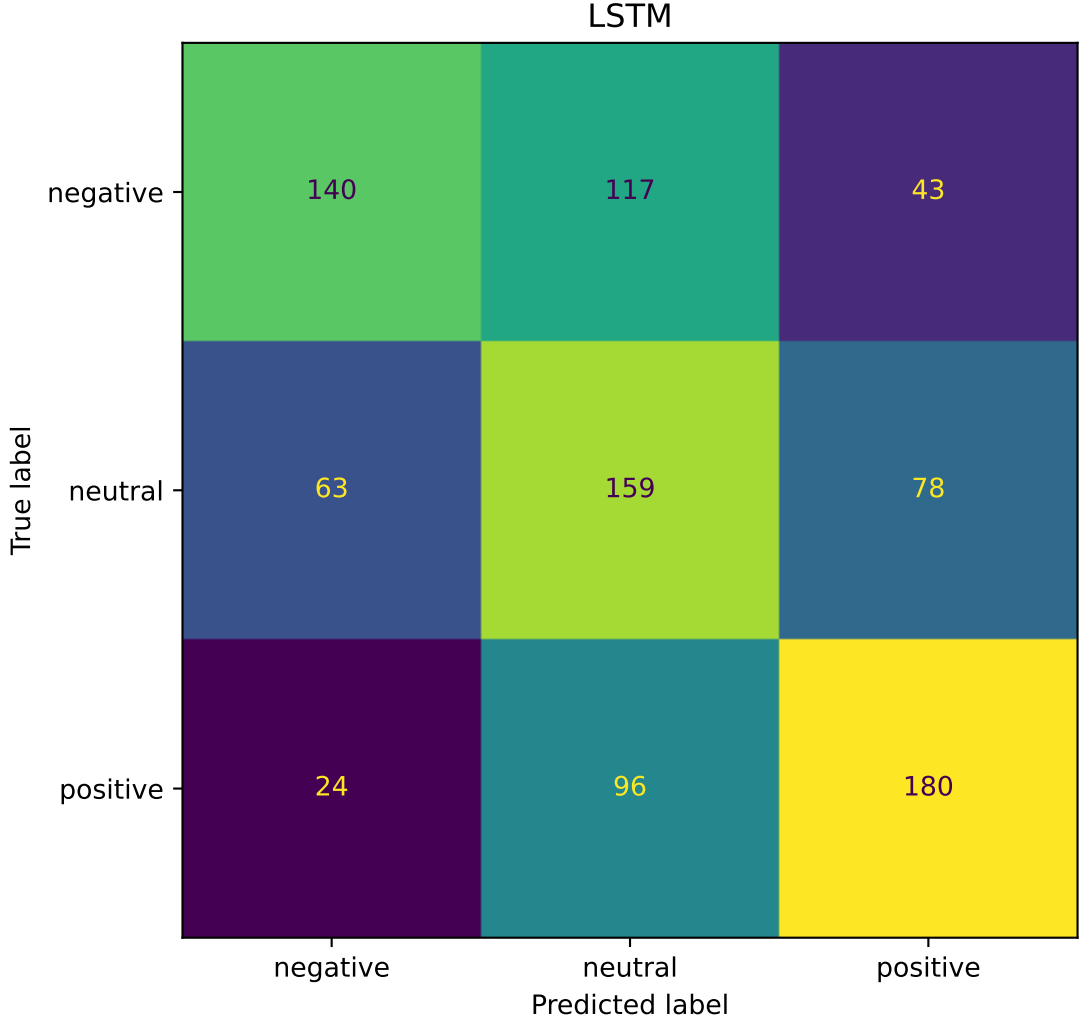
Held-out test comparison

Model	Accuracy	Macro P	Macro R	Macro F1	Fit + val (s)	ms/review
BERT-Mini	0.606	0.605	0.606	0.605	115.1	1.03
LSTM	0.532	0.547	0.532	0.535	29.6	0.15

Timing: one warmed-up batched test pass on the same device.
Includes batch transfers and prediction collection; excludes tokenization/model download.
ms/review is amortized throughput, not single-request latency. Balanced chance accuracy = 1/3.


Learning curves
Training cross-entropy
Epoch
BERT-Mini
LSTM
1.1
1.0
0.9
0.8
0.7
0.6
0.5
0.4
1
2
3
4
5
6
7
8
Checkpoint selection
Validation macro F1
Epoch
BERT-Mini
LSTM
0.65
0.60
0.55
0.50
0.45
0.40
0.35
0.30
1
2
3
4
5
6
7
8


BERT-Mini
negative
neutral
positive
True label
183
96
21
85
142
73
16
64
220
negative
neutral
positive
Predicted label


LSTM
negative
neutral
positive
True label
140
117
43
63
159
78
24
96
180
negative
neutral
positive
Predicted label

BERT-Mini: per-class test results

	precision	recall	f1-score	support
negative	0.644	0.610	0.627	300
neutral	0.470	0.473	0.472	300
positive	0.701	0.733	0.717	300
accuracy			0.606	900
macro avg	0.605	0.606	0.605	900
weighted avg	0.605	0.606	0.605	900

LSTM: per-class test results

	precision	recall	f1-score	support
negative	0.617	0.467	0.531	300
neutral	0.427	0.530	0.473	300
positive	0.598	0.600	0.599	300
accuracy			0.532	900
macro avg	0.547	0.532	0.535	900
weighted avg	0.547	0.532	0.535	900

BERT: observed test errors

True: negative; predicted: neutral. Review: The product came in a plastic bag and box but the labels were wet so I could not read them. I sprayed the product and there was very little change on the leads but it pretty much hurt all the plants.

True: neutral; predicted: positive. Review: Great must-have tool! A long time ago I was able to open some of my watches, but some were so tight, that it was almost impossible to avoid scratches. This tool turns this job into a few second fun.

True: positive; predicted: neutral. Review: I'm not fond if the textured backdrops. Otherwise it works ok and for the price it's ok but a smooth surface would be better.

True: negative; predicted: positive. Review: Well made, but needs to have a larger handle to hold really small.

These are the first errors in test order, not hand-picked successes. Rating labels, mixed sentiment, sarcasm and review truncation can all cause disagreement; an error example alone does not prove its cause.

Transfer learning and efficiency

Observed test macro F1: BERT-Mini=0.605; LSTM=0.535; difference=+0.071. This is a single seeded comparison on the included sample, not proof that either architecture always wins.

BERT begins with language representations learned during pretraining; fine-tuning adapts them using fewer task-specific labels than learning language from random weights. The LSTM learns its embeddings and sequence weights from this training set. Both reuse the fixed pretrained tokenizer. A random-initialized BERT control and multiple sample sizes would be needed to isolate and quantify the benefit of pretraining itself.

BERT-Mini has 11,171,331 trainable parameters; LSTM has 1,986,883. FP32 weight storage estimates: 42.6 MiB vs 7.6 MiB. These are parameter-only estimates, not measured RAM. Gradients, AdamW optimizer states, activations and the runtime require additional memory.

Measured fit + validation: 115.1s for BERT-Mini (4 epochs); 29.6s for LSTM (8 epochs). Different epoch budgets and parameter counts make this a practical baseline comparison, not an equal-compute experiment. Offline pretraining cost and first-time downloads are excluded.

BERT uses bidirectional self-attention; attention cost grows quadratically with sequence length. LSTM processes tokens sequentially and has fewer parameters here. Shortening the token limit saves work but may discard important sentiment near the end of a review.

Model source and architecture: https://huggingface.co/google/bert_uncased_L-4_H-256_A-4. Turc et al. (2019), Well-Read Students Learn Better, https://arxiv.org/abs/1908.08962.

Deployment constraints and reproduction

Latency and resources: measure end-to-end p50/p95 latency on the target CPU/GPU including tokenization, queueing and model load. Batch throughput reported here is not a serving SLA. Cache one loaded model and tokenizer; test quantization/distillation for smaller deployments before accepting any accuracy tradeoff.

Input limits: this is an English-only sample with a 96-token limit. Long reviews, other languages, sarcasm and new product categories need separate evaluation. Original source splits do not guarantee product- or reviewer-disjoint evaluation because those identifiers are absent from the mirror. Balanced sampling does not reflect live class priors.

Reliability: monitor per-class F1 and changing language/product patterns. Validate and calibrate probabilities on fresh data before using a confidence threshold. Send uncertain/high-impact cases for human review. Do not treat an uncalibrated softmax as trustworthy certainty.

Privacy: minimize review logging, redact personal information, set retention limits and restrict access. Package exact model/tokenizer revisions and tested dependency versions; assess dataset/model terms for the intended use.

Execution: Darwin 27.0.0, arm64, device=mps, CPU threads=4. Python 3.12.16; torch 2.15.0; transformers 4.57.6; pandas 3.1.0; scikit-learn 1.9.1; Matplotlib 3.11.2. Seeds fix sampling and initialization; hardware kernels can still vary.

Reproduce: install packages listed at the top of code.py, place dataset.csv beside it, then run python code.py. Training writes only output.pdf; the trained models stay in memory. predict_review() demonstrates inference. A production service would separately persist and version trained weights.

HuggingFace fine-tuning reference: https://huggingface.co/docs/transformers/tasks/sequence_classification. PyTorch LSTM: https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html